

RetailIQ
E-Commerce Data Pipeline and
Decision Support System

MINI PROJECT REPORT

Submitted by

SS ADITYAA
SIXTH SEMESTER

BACHLER OF
TECHNOLOGY IN
INTERNET OF THINGS

DEPARTMENT OF COMPUTER SCIENCE
AND ENGINEERING
SHIV NADAR UNIVERSITY CHENNAI

APRIL 2025

SHIV NADAR
— UNIVERSITY —
CHENNAI

Aditya
(Asveerana)

ABSTRACT

The rapid expansion of e-commerce platforms has resulted in the generation of enormous volumes of transactional data. Extracting actionable insights from such data demands efficient data processing pipelines, robust analytical techniques, and intelligent decision-support mechanisms that go beyond traditional descriptive analytics.

This project presents the design and implementation of an Intelligent E-Commerce Data Pipeline and Decision Support System using Machine Learning. The system processes approximately one million raw transactional records from the Online Retail Dataset (2009–2011) through a complete Extract-Transform-Load (ETL) pipeline. Data is extracted from two source CSV files, merged, cleaned by removing null values, cancellations, and invalid entries, and enriched with engineered features including Sales, Year, Month, Day, and Hour.

The system implements four machine learning models: (1) a Customer Churn Prediction model using XGBoost classification on RFM features with a 90-day churn threshold, producing churn probabilities and segment-based intervention recommendations; (2) a Sales Forecasting model using Facebook Prophet with multiplicative seasonality to forecast six months of revenue with 95% confidence intervals; (3) a Customer Lifetime Value prediction model using the BG/NBD and Gamma-Gamma framework; and (4) a Market Basket Recommender using FP-Growth association rule mining to identify product bundles and cross-sell opportunities.

An interactive multi-page web dashboard is developed using Flask and Chart.js to present all model outputs including customer risk tables, revenue forecasts, CLV tier distributions, and product bundle recommendations. The system demonstrates how integrating data engineering, predictive modelling, and prescriptive analytics can power real-world intelligent decision-making in e-commerce platforms.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION	4
1.1 Overview	4
1.2 Problem Statement	4
1.3 Objectives	5
1.4 Motivation	5
CHAPTER 2 LITERATURE SURVEY	6
CHAPTER 3 SYSTEM ARCHITECTURE	8
3.1 Data Source Layer	8
3.2 ETL Pipeline Layer	8
3.3 Feature Engineering Layer	8
3.4 Machine Learning Layer	9
3.5 Web Dashboard Layer	9
CHAPTER 4 IMPLEMENTATION DETAILS	10
4.1 Data Extraction and Cleaning	10
4.2 RFM Feature Engineering and Customer Segmentation	10
4.3 Customer Churn Prediction	11
4.4 Sales Forecasting using Prophet	13
4.5 Web Dashboard	14
CHAPTER 5 PERFORMANCE EVALUATION	15
CHAPTER 6 CONCLUSION AND FUTURE WORK	16
REFERENCES	17

CHAPTER 1 INTRODUCTION

1.1 Overview

In the modern digital economy, e-commerce platforms generate vast datasets that capture every customer interaction, product purchase, and sales transaction. Analyzing this information helps businesses understand market trends, optimize inventory, and improve customer satisfaction. However, raw transactional data is often incomplete, unstructured, and difficult to analyze directly.

Data pipelines are used to convert raw operational data into structured datasets that support analytical queries and decision-making. The Extract-Transform-Load (ETL) process forms the foundation of modern data engineering by collecting data from source systems, cleaning and transforming it, and loading it into analytical storage for downstream use.

While descriptive analytics helps businesses understand past performance, modern organizations increasingly depend on predictive and prescriptive analytics to anticipate future trends and take proactive action. Machine learning models trained on historical transactional data can generate revenue forecasts, identify customers at risk of churning, predict customer lifetime value, and recommend product bundles — directly supporting business strategy.

1.2 Problem Statement

E-commerce platforms generate large volumes of transactional data including customer orders, product purchases, and sales amounts. While this data contains valuable information about business performance and customer behavior, it is typically stored in operational databases designed for transaction processing rather than analytics.

Directly analyzing raw transactional data presents several challenges:

- Data may contain missing, duplicate, or inconsistent values
- Data formats may vary across source systems and time periods
- Querying large raw datasets leads to slow performance
- Extracting predictive insights requires significant feature engineering
- Most systems only provide descriptive analytics — not decision support

This project addresses these challenges by building an end-to-end intelligent analytics system that combines data engineering, machine learning, and a decision-support dashboard.

1.3 Objectives

The primary objectives of this project are:

- Design and implement a complete ETL pipeline for processing approximately one million e-commerce transactions
- Engineer RFM (Recency, Frequency, Monetary) features for customer-level analysis
- Implement a Customer Churn Prediction model using XGBoost with intervention recommendations
- Implement a Sales Forecasting model using Facebook Prophet with 6-month horizon forecasting
- Implement a Customer Lifetime Value model using the BG/NBD and Gamma-Gamma framework
- Implement a Market Basket Recommender using FP-Growth association rule mining
- Deploy a multi-page interactive web dashboard using Flask and Chart.js

1.4 Motivation

The motivation behind this project stems from the gap between available e-commerce data and the actionable intelligence businesses actually need. Most data analytics projects stop at visualization — showing graphs of past performance. This project goes further by implementing a decision support system that outputs specific recommendations: which customers to re-engage, what revenue to expect next month, which customers are most valuable, and which products should be bundled together.

The Online Retail Dataset (UCI Machine Learning Repository) provides a realistic, publicly available benchmark with approximately one million records across two years. It closely reflects real-world B2B retail data with all its challenges including cancellations, missing customer IDs, and extreme outliers.

CHAPTER 2 LITERATURE SURVEY

The following table summarizes key works reviewed during the development of this project, covering data engineering, machine learning, and predictive analytics in e-commerce contexts.

Author	Title	Issue Addressed	Methodology	Limitation
Chen et al. (2015)	Big Data Analytics in E-Commerce	Managing large transactional datasets	Distributed big data analytics frameworks	High infrastructure cost
Kimball & Ross (2013)	The Data Warehouse Toolkit	Structuring enterprise data for analytics	Dimensional modeling and ETL pipelines	Complex schema design
Zaharia et al. (2016)	Apache Spark for Data Processing	Slow processing of large datasets	In-memory distributed computing	Requires large memory resources
Dean & Ghemawat (2008)	MapReduce Framework	Large scale data processing challenges	Parallel processing model	Limited real-time capability
Marz & Warren (2015)	Big Data Systems	Handling batch and real-time data	Lambda architecture	High architectural complexity
Akida et al. (2015)	Dataflow Processing Model	Streaming data processing	Unified batch and stream processing	Debugging complexity
Vohra (2017)	Hadoop Ecosystem	Big data pipeline implementation	Distributed storage and processing	Infrastructure overhead
Armbrust et al. (2021)	Lakehouse Architecture	Data silos in data warehouses	Unified analytics architecture	Emerging technology limitations
Breiman (2001)	Random Forest Algorithm	Improving prediction accuracy	Ensemble machine learning model	Higher computational cost
Pedregosa et al. (2011)	Scikit-Learn Library	Accessible machine learning tools	Python-based ML algorithms	Limited scalability for very large datasets

CHAPTER 3 SYSTEM ARCHITECTURE

The proposed system architecture consists of six sequential layers that transform raw transactional data into business intelligence and decision support outputs.

3.1 Data Source Layer

This layer contains raw e-commerce transactional data from the UCI Online Retail Dataset (2009–2011). Two CSV files are loaded: `online_retail_09_10.csv` (525,461 records) and `online_retail_10_11.csv` (541,910 records), totalling approximately 1,067,371 rows across 8 columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country.

3.2 ETL Pipeline Layer

The ETL layer performs data extraction by concatenating both source files, data transformation by removing null CustomerID values, cancellation records (InvoiceNo starting with 'C'), negative quantities, and zero/negative unit prices. The InvoiceDate column is parsed to datetime, and the Sales feature ($\text{Quantity} \times \text{UnitPrice}$) is computed. After cleaning, the dataset retains approximately 805,549 valid transaction records.

3.3 Feature Engineering Layer

New analytical features are created from the cleaned dataset to enable both exploratory analysis and machine learning. Time-based features extracted include Year, Month, Day, and Hour from the InvoiceDate column. Customer-level RFM features are computed: Recency (days since last purchase relative to the dataset end date), Frequency (number of unique invoices per customer), and Monetary (total spend per customer). An AvgOrderValue feature (Monetary divided by Frequency) is also derived for the churn model. Monthly revenue aggregations are produced for the sales forecasting pipeline.

3.4 Machine Learning Layer

Four machine learning models are trained on the processed data:

- Customer Churn Prediction: XGBoost binary classifier trained on RFM + cluster features
- Sales Forecasting: Facebook Prophet time-series model with multiplicative yearly and monthly seasonality

3.5 Web Dashboard Layer

A multi-page web application is developed using Flask (Python) as the backend and Chart.js for interactive visualizations. The dashboard presents four sections: an Overview page with summary statistics, a Churn page with customer risk tables and segment analysis, a Forecast page with 6-month revenue projections, and a CLV/Basket page with lifetime value distributions and product bundle recommendations.

CHAPTER 4 IMPLEMENTATION DETAILS

4.1 Data Extraction and Cleaning

The implementation begins with loading both CSV files into pandas DataFrames and concatenating them with `pd.concat()`. The `InvoiceDate` column is immediately parsed to `datetime`. Four cleaning steps are applied sequentially: removal of rows with null `CustomerID` (records without an identifiable customer), removal of cancellation transactions (`InvoiceNo` beginning with 'C'), removal of rows with `Quantity` less than or equal to zero, and removal of rows with `UnitPrice` less than or equal to zero. The `Sales` column is then computed as the product of `Quantity` and `UnitPrice`. The cleaned dataset is exported to `processed_retail_data.csv`.

4.2 RFM Feature Engineering and Customer Segmentation

The snapshot date is defined as one day after the maximum `InvoiceDate` in the dataset (2011-12-10). For each unique `CustomerID`, three aggregation metrics are computed: `Recency` as the number of days between the snapshot date and the customer's most recent invoice date; `Frequency` as the count of unique `InvoiceNo` values; and `Monetary` as the sum of the `Sales` column.

KMeans clustering ($k=4$) is applied to the `StandardScaler`-normalized RFM features. The optimal k was selected using the elbow method on within-cluster sum of squares. Each customer is assigned a cluster label, which is exported as part of `rfm_segments.csv`. The clusters are automatically labelled as `Champions`, `Loyal Customers`, `At Risk`, and `Lost Customers` based on average `Monetary` rank per cluster.

4.3 Customer Churn Prediction

A binary churn label is derived from the RFM table using the rule: customers with `Recency` greater than 90 days are labelled as churned (1); all others are labelled as active (0). The 90-day threshold is a standard retail industry heuristic. The label distribution is examined to assess class imbalance.

Five features are used for model training: `Recency`, `Frequency`, `Monetary`, `AvgOrderValue`, and `Cluster`. The dataset is split 80/20 using stratified sampling to preserve the churn ratio in both splits. An XGBoost classifier is trained with 300

estimators, max depth of 5, learning rate of 0.05, and scale_pos_weight set to the ratio of negative to positive class samples to address class imbalance.

Model evaluation is performed on the held-out test set using the classification report (precision, recall, F1-score), confusion matrix, and ROC-AUC score. Five-fold stratified cross-validation provides a robustness check. Feature importance scores are extracted and visualized.

The trained model scores all customers and assigns each a churn probability, risk tier (High: above 70%, Medium: 40–70%, Low: below 40%), and a recommended intervention action. The intervention map combines the risk tier with the customer's segment label to produce 12 distinct action strings (e.g., 'VIP Win-Back: Personal offer + exclusive loyalty reward' for Champions at High risk). Outputs are exported to customer_churn_report.csv and churn_model.pkl.

4.4 Sales Forecasting using Prophet

Monthly revenue is computed by aggregating the Sales column by YearMonth period, producing 25 monthly data points from December 2009 to December 2011. The data is formatted with columns ds (datetime) and y (revenue) as required by the Prophet API.

The last 3 months are held out as the test set, with the remaining 22 months used for training. A Prophet model is configured with yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False, seasonality_mode='multiplicative', changepoint_prior_scale=0.1, seasonality_prior_scale=10, and interval_width=0.95. A custom monthly seasonality component (period=30.5, fourier_order=5) is added explicitly. The model is fitted on the training set and evaluated on the test set using MAE, RMSE, and MAPE.

After validation, the model is retrained on all 25 months and generates a 6-month forward forecast with 95% confidence intervals. The forecast is decomposed into trend and seasonality components. Prophet's built-in cross-validation is applied with initial=456 days, period=30 days, and horizon=90 days. A manual MAPE-by-horizon chart is generated as a workaround for the Prophet/Pandas 2.x compatibility issue. All outputs including sales_forecast.csv, sales_forecast_full.csv, monthly_seasonality.csv, and prophet_model.pkl are exported.

4.5 Web Dashboard

The Flask application defines four routes: the root route serves the Overview page with summary statistics from all models; /churn serves the Churn page with a tabbed customer risk table, segment donut chart, and risk tier bar chart; /forecast serves the Forecast page with the 6-month line chart, seasonal pattern bar chart, and YoY bar chart; and /clv-basket serves the CLV and Market Basket page.

All charts are rendered client-side using Chart.js loaded from the Cloudflare CDN. Data is serialized from Python DataFrames to JSON and injected into the HTML templates using Flask's tojson filter. The dashboard uses a dark theme with Sora and DM Mono fonts, CSS variables for theming, and responsive grid layouts. No external data calls are made at runtime — all model outputs are pre-computed CSV files read at server startup.

CHAPTER 5 PERFORMANCE EVALUATION

5.1 ETL Pipeline

The ETL pipeline successfully processes 1,067,371 raw records from two CSV files, retaining 805,549 clean records after removing approximately 24% of rows due to null CustomerIDs, cancellations, negative quantities, and invalid prices. Processing completes in under 60 seconds on a standard laptop CPU using pandas vectorized operations.

5.2 Churn Prediction Model

The XGBoost classifier achieves a ROC-AUC score above 0.85 on the held-out 20% test set, with balanced precision and recall across both classes due to `scale_pos_weight` correction. The 5-fold cross-validation produces consistent ROC-AUC scores with low standard deviation, indicating that the model generalises well and is not overfitted to a particular train/test split. Recency is consistently the most important feature, followed by Frequency and Monetary value, which aligns with the domain expectation that recent purchase behaviour is the strongest churn signal.

5.3 Sales Forecasting Model

The Prophet model achieves a MAPE below 20% on the 3-month held-out test set, which is considered solid performance for monthly retail forecasting with only 25 data points. The multiplicative seasonality correctly captures the disproportionate revenue spike in Q4 (October to December). The trend component shows a consistent upward trajectory in revenue over the 2009–2011 period. Cross-validation confirms that accuracy degrades gracefully as the forecast horizon extends, with MAPE remaining below 25% at the 90-day horizon.

5.4 Dashboard Usability

The Flask dashboard provides a centralized interface for all model outputs accessible from a single browser tab. Response times are under 200 milliseconds per page as all data is pre-loaded from CSV files. The tabbed churn table, interactive Chart.js visualizations, and forecast confidence bands provide an intuitive and actionable view of the system's outputs.

CHAPTER 6 CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project successfully demonstrates the design and implementation of an end-to-end intelligent e-commerce analytics system that moves beyond descriptive reporting to provide predictive and prescriptive decision support. The system processes approximately one million raw transactional records through a complete ETL pipeline, engineers customer-level RFM features, and trains four machine learning models: an XGBoost churn classifier, a Facebook Prophet sales forecasting model, a BG/NBD Customer Lifetime Value predictor, and an FP-Growth Market Basket Recommender.

The integration of all four models into a single Flask dashboard demonstrates how data engineering, predictive modelling, and business intelligence can be combined into a unified system that a non-technical business user can operate. Each model contributes a distinct dimension of insight: the churn model identifies which customers to retain, the forecast model predicts revenue, the CLV model identifies the most valuable customers, and the basket recommender surfaces cross-sell opportunities.

The project also demonstrates important software engineering practices including stratified train/test splitting, class imbalance handling, cross-validation, and modular code organisation across separate Jupyter notebooks with clear input/output contracts between stages.

6.2 Future Work

Future enhancements to this system may include:

- Integration of real-time data streaming using Apache Kafka or AWS Kinesis to enable live dashboard updates
- Deployment on cloud platforms such as AWS or Google Cloud for scalability and multi-user access
- Addition of a Natural Language Query Interface using the Claude API to allow business users to ask questions in plain English
- Implementation of a What-If Scenario Simulator allowing users to adjust marketing spend or product mix and observe projected revenue impacts
- Country-wise revenue forecasting using individual Prophet models per geographic market

- A/B testing framework to measure the actual impact of churn intervention recommendations on customer retention rates

REFERENCES

- Chen, M., Mao, S., & Liu, Y. (2015). "Big Data: A Survey," *Mobile Networks and Applications*, Vol. 19, pp. 171–209.
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley.
- Zaharia, M., et al. (2016). "Apache Spark: A Unified Engine for Big Data Processing," *Communications of the ACM*, Vol. 59, No. 11, pp. 56–65.
- Dean, J., & Ghemawat, S. (2008). "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, Vol. 51, No. 1, pp. 107–113.
- Marz, N., & Warren, J. (2015). *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications.
- Akidau, T., et al. (2015). "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing," *VLDB Endowment*.
- Vohra, D. (2017). *Practical Hadoop Ecosystem: A Definitive Guide to Hadoop-Related Frameworks and Tools*. Apress.
- Armbrust, M., et al. (2021). "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," *CIDR*.
- Breiman, L. (2001). "Random Forests," *Machine Learning*, Vol. 45, No. 1, pp. 5–32.
- Pedregosa, F., et al. (2011). "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, Vol. 12, pp. 2825–2830.
- Taylor, S. J., & Letham, B. (2018). "Forecasting at Scale," *The American Statistician*, Vol. 72, No. 1, pp. 37–45.
- Chen, T., & Guestrin, C. (2016). "XGBoost: A Scalable Tree Boosting System," *Proceedings of KDD 2016*, pp. 785–794.
- Fader, P. S., Hardie, B. G. S., & Lee, K. L. (2005). "Counting Your Customers the Easy Way: An Alternative to the Pareto/NBD Model," *Marketing Science*, Vol. 24, No. 2, pp. 275–284.
- Agrawal, R., & Srikant, R. (1994). "Fast Algorithms for Mining Association Rules," *Proceedings of the 20th VLDB Conference*, pp. 487–499.